

Active Liquidity Framework - DualPool



Uniswap

July 14, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	4
Scope	5
System Overview	7
Refactored Code	9
Security Model and Trust Assumptions	9
Privileged Roles	9
Trust Assumptions	10
Additional Considerations	11
Medium Severity	13
M-01 removeLiquidity Withdrawals Ignore ERC-6909 Claims At Payout	13
M-02 Stale Transient Liquidity Slots Across Multiple Swaps in One Transaction Can Revert the Swap	15
M-03 Fee-Charging ERC-4626 Vaults Break Share Accounting on Deposit and Withdrawal	16
M-04 A Dust _afterSwap Deposit To Vault Can Revert an Entire Swap via ZeroSharesMinted	17
M-05 Vault Deposit Cap Can Cause Swaps To Revert During _afterSwap Re-Deposit	18
M-06 sqrtPrice Can Be Manipulated for Free in a Live-but-Unbootstrapped Pool	19
Low Severity	20
L-01 Inconsistent Native Currency Handling in SettlementLib	20
L-02 initializePool Accepts Dynamic-Fee Keys, Permanently Creating a Pool Where SmartPoolHook Cannot Operate Correctly	21
L-03 Redeeming Accumulated Claims Within the Same Transaction Can Revert Later Swaps	22
L-04 Fixed _decimalsOffset = 12 Makes 6-Decimal Pairs Impractical To Bootstrap	24
L-05 renounceOwnership Allows Permanent Loss Of Administrative Control	25
L-06 Documentation Does Not Match Implementation in Several Places	26
L-07 Donated Vault Shares Inflate the Pro-Rata Denominator and Suppress Reported Effective Liquidity	27
L-08 getIndicativeQuote May Overstate Output Beyond Pool-Deliverable Reserves	29
Notes & Additional Information	30
N-01 Duplicate Code	30
N-02 Unnamed Magic Number in SmartPoolHook	30
N-03 Vault Addresses Are Permanently Assigned With No Emergency Revocation Path	31
N-04 Allowance Mutation Bypasses InventoryLib in DualPoolHook.refreshVaultApproval	32
N-05 Overloaded "bucket" Terminology	33
N-06 Inconsistent Parameter Order in earned	34
Conclusion	35

Appendix	36
Issue Classification	36

Summary

Type	DeFi	Total Issues	20 (19 resolved)
Timeline	From 2026-05-20 To 2026-06-17	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	6 (6 resolved)
		Low Severity Issues	8 (7 resolved)
		Notes & Additional Information	6 (6 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the [Uniswap/v4-hooks-internal](#) repository at commit [28e8e48](#).

In scope were the following files:

```
src
├── alf
│   ├── base
│   │   ├── vault
│   │   │   ├── MultiAssetShareMath.sol
│   │   │   └── MultiAssetVault.sol
│   │   ├── JITLockable.sol
│   │   ├── PoolVault.sol
│   │   └── SmartPoolBase.sol
│   ├── interfaces
│   │   └── IALFHook.sol
│   └── SmartPoolHook.sol
```

As a subsequent request due to the code being refactored we audited the same repository at commit [2f5fab5](#). In scope were the following files:

```
src
├── alf
│   ├── base
│   │   ├── vault
│   │   │   └── MultiAssetShareMath.sol
│   │   ├── OwnedALFHook.sol
│   │   └── PoolVault.sol
│   ├── interfaces
│   │   ├── IALFHook.sol
│   │   └── IHookStats.sol
│   ├── libraries
│   │   ├── FeeLib.sol
│   │   ├── InventoryLib.sol
│   │   └── SettlementLib.sol
│   ├── types
│   │   ├── ActiveLiquidity.sol
│   │   ├── DepositGate.sol
│   │   ├── Distribution.sol
│   │   ├── Inventory.sol
│   │   ├── JITLock.sol
│   │   ├── Liveness.sol
│   │   ├── Rewards.sol
│   │   └── Shares.sol
```

- └─ DualPoolHook.sol
- └─ DualPoolIncentivizedHook.sol

System Overview

The Active Liquidity Framework components in scope implement three related capabilities on top of Uniswap v4. [SmartPoolHook](#) provides a managed just-in-time (JIT) liquidity strategy in which capital is accounted for as internal pool shares, kept partly off-pool between swaps, and deployed into concentrated liquidity ranges only during swap execution. [ALFMultiplexer](#) provides a routing layer, exposed as a virtual zero-liquidity pool, that forwards a single swap across one or more targeted ALF hooks and returns the aggregated result to the caller. [SimpleSpreadQuoterHook](#) provides a static-fee spread quoter built on native v4 liquidity concentrated in a single tick-spacing-wide range, exposing indicative quotes through the [IALFHook](#) interface so that it can be consumed as a candidate by [ALFMultiplexer](#) or queried directly by off-chain routers.

The [SmartPoolHook](#) stack is organized around three main layers. [MultiAssetVault](#) and [MultiAssetShareMath](#) implement the core two-asset share accounting model, where users hold non-transferable internal shares representing a proportional claim on a pool's managed assets. [PoolVault](#) adapts that accounting to Uniswap v4 by tracking each pool's assets across raw ERC-20 balances, PoolManager ERC-6909 claims, and optional ERC-4626 vault positions. [SmartPoolHook](#) then adds the swap-time strategy itself, initializing pools, accepting liquidity deposits and withdrawals, computing a weighted distribution across tick ranges, deploying liquidity in [beforeSwap](#), and removing it again in [afterSwap](#).

A pool's reserves are therefore not limited to standing liquidity inside the Uniswap pool. Between swaps, assets may remain as tracked ERC-20 balances, as ERC-6909 claims awaiting redemption, or as ERC-4626 vault shares. For quoting and JIT deployment, the implementation distinguishes between total economic assets and the subset that can be immediately withdrawn from configured vaults. This distinction is important to the security model because LP share accounting uses total managed value, while swap execution depends on current vault withdrawability.

JIT lifecycle ([SmartPoolHook](#), [JITLockable](#)): pools are priced by a static fee taken from [PoolKey.fee](#); the hook intentionally ignores swap [hookData](#). On [beforeSwap](#), the hook acquires a per-pool transient lock, computes the liquidity each owner-configured tick bucket should receive from the pool's current effective assets and weights, redeems any outstanding ERC-6909 claims, withdraws only the shortfall from the configured vaults, and deploys each bucket as a concentrated LP position. The v4 swap then executes against that deployed

liquidity. On `afterSwap`, the hook removes all positions, resolves its net `PoolManager` delta for each currency (settling amounts it owes from its ERC-20 balance and minting surplus as ERC-6909 claims because the swapper has not yet settled), re-deposits leftovers to the vaults, and clears the lock. `JITLockable` provides both a per-pool lock and a global in-flight counter; the per-pool lock prevents a corrupted lifecycle from a same-pool reentry, and the global counter lets external user and admin entry points reject any call that originates inside an in-flight JIT cycle. User-facing entry points additionally carry a transient `nonReentrant` guard.

Share accounting (`PoolVault`, `MultiAssetVault`, `MultiAssetShareMath`): deposits mint non-transferable internal shares. The owner seeds a pool through `bootstrap`, which mints `sqrt(amount0 * amount1)` shares in the Uniswap V2 style and fixes the initial share-to-asset ratio. Conversions use the EIP-4626 virtual-offset pattern (adding virtual assets and virtual shares to the math) to resist share-price inflation attacks, and bootstrap is rejected below a minimum-share floor because seeds smaller than the virtual position would permanently forfeit a meaningful fraction of capital to it. `PoolVault` tracks three asset sources per pool and currency: ERC-4626 vault shares, ERC-6909 `PoolManager` claims, and raw ERC-20 attributed to the pool, all isolated per pool so that pools sharing a vault or currency do not consume each other's balances. Deposits round up and withdrawals round down, and a withdrawal in the same block as the depositor's last deposit is rejected to defend against atomic deposit-swap-withdraw sniping.

A deliberate accounting asymmetry governs how balances are read. Share math reads total assets uncapped (including capital momentarily locked in a paused or constrained vault) so that LP exit value reflects the pool's full economic stake, while JIT deployment sizing and indicative quotes read an effective balance that caps the vault contribution at this pool's pro-rata share of what the vault can actually pay out right now.

Metadata base and indicative quoting (`SmartPoolBase`, `IALFHook`): `SmartPoolBase` supplies the `IALFHook` metadata surface (liveness, declared gas budget, ERC-165 advertisement) and `DeltaResolver` settlement. The base returns a zero indicative quote, and `SmartPoolHook` overrides it to quote against hypothetical JIT liquidity.

This indicative path differs in a security-relevant way from how a swap actually executes in Uniswap v4, and from the `SwapSimulator` used elsewhere in the framework. The real v4 swap loop walks ticks: as price moves, it crosses tick boundaries, updates active liquidity at each crossing, and continues stepping until the amount is exhausted or the price limit is reached. `SmartPoolHook`'s indicative quote does not do this. It sums a single constant liquidity figure from only the buckets whose range contains the current tick, each pre-budgeted by its weight, and then performs a single swap-step computation at the current price. It does

not walk ticks, does not cross bucket boundaries, and does not account for out-of-range buckets activating as price moves through them during a real multi-range swap. As a result, the indicative is representative only for swaps small enough to remain within the current tick's in-range liquidity; for larger swaps that would move price across bucket or tick boundaries at execution, the quote can diverge from the amount the JIT cycle actually delivers. The quote is also computed against effective (cap-aware) assets and the static fee, and reflects liquidity that is only hypothetically deployed at quote time rather than liquidity that currently sits in the pool. Routers and aggregators should treat the value as a non-binding hint and rely on execution-time slippage controls, not on the indicative, for correctness.

Refactored Code

The code related to the `DualPoolHook` has been refactored into a more type-centered structure, in which logic that previously lived in monolithic base contracts is now organized into self-contained types (for example, `Inventory`, `Shares`, `JITLock`, `Liveness`, and `Distribution`) and dedicated libraries (`InventoryLib`, `SettlementLib`, and `FeeLib`). This reorganization is structural: the underlying behavior of the hook — its JIT spread quoting, ERC-4626 rehypothecation, share accounting, and settlement flows — remains functionally equivalent to the previously reviewed implementation.

In addition, a new `DualPoolIncentivizedHook` contract has been introduced. It extends the `DualPoolHook` with a per-block liquidity-incentives program that allows LP shareholders to earn a separate reward token, without modifying the underlying swap and vault logic inherited from the base hook.

Security Model and Trust Assumptions

Privileged Roles

- `owner` of `SmartPoolHook`
 - Initializes pools, including their initial price, distribution, deposit permissions, and configured ERC-4626 vaults.
 - Bootstraps pools, which establishes the initial share-to-asset ratio for LP accounting.

- Updates liquidity distributions, toggles whether a pool is live for swaps, and controls whether external accounts may add liquidity.
- Can refresh token allowances to configured vaults.

Trust Assumptions

- Configured ERC-4626 vaults are trusted dependencies.
 - `PoolVault` grants vaults large standing token allowances and values vault positions using `convertToAssets`. A malicious, compromised, or misleading vault can affect withdrawability, distort reserve accounting, or potentially endanger balances of a shared currency held by the hook.
- The bootstrap process must be performed with sufficiently large seed capital.
 - `MultiAssetVault` uses virtual shares and virtual assets to mitigate post-bootstrap inflation attacks, but the same mechanism creates permanent bootstrap drift when the initial share supply is too small. The implementation enforces a minimum bootstrap floor, yet operators still need to choose bootstrap sizes that are economically appropriate for the asset pair.
- Native `ETH` is not supported.
 - Pool initialization rejects an `address(0)` currency, so operators must use a wrapped variant such as WETH.
- Fee-on-transfer and rebasing tokens are not supported.
 - The deposit path mints shares against the requested amounts and reverts with `TransferReceiptShortfall` on any under-receipt, and per-pool holdings are tracked as a recorded figure rather than live `balanceOf`, so rebases are not reflected. Operators must use standard ERC-20s.
- Pool currencies should not support callbacks.
 - Pool currencies must not invoke external/attacker-reachable code on transfer (no callback/hook tokens). The per-pool JIT lock blocks same-pool reentrancy but deliberately permits a swap on one pool to nest inside another pool's in-flight JIT cycle (cross-pool swap reentrancy is not prohibited). A currency whose transfer (or take/settle) hands control to arbitrary code could therefore reenter and swap a second pool sharing the same hook mid-cycle, corrupting cross-pool delta

attribution (e.g. the non-pool-scoped currencyDelta read in `_resolveNetDeltaCurrency`) and the `maxWithdraw`-based JIT funding. Operators must restrict pool currencies to plain ERC-20s without transfer callbacks; tokens with callback semantics (e.g. ERC-777 or bespoke hook-on-transfer tokens) must not be onboarded.

- ERC-4626 withdrawal caps
 - ERC-4626 vault `maxWithdraw` may be liquidity-capped (e.g. Spark spUSDT, Morpho V2 limit `maxWithdraw` / `maxRedeem` by available liquidity, not share balance) and is mutable by the vault operator. The hook correctly under-sizes JIT and quotes against `maxWithdraw`-capped effective assets, but as a consequence LP withdrawals and JIT funding for that currency can temporarily revert (`VaultLiquidityShortfall`) while the vault is illiquid, even though LP economic claims (total assets) are unaffected. Operators must use vaults whose liquidity profile and governance over `maxWithdraw` are acceptable.
- LP depositors and market makers.
 - Addresses that deposit through `bootstrap` or `addLiquidity` hold non-transferable internal shares. Their capital is deployed and withdrawn automatically by the JIT lifecycle and may be parked in operator-chosen vaults between swaps. Withdrawal through `removeLiquidity` is always available regardless of pool liveness, slippage bounds and a deadline are enforced after the actual transfers, and a same-block deposit-then-withdraw is blocked. Depositors must trust the owner not to disable external deposits, reshape the distribution, or configure vaults in ways that disadvantage them, and must seed or deposit at sizes above the inflation-defense floor to avoid forfeiting capital to the virtual position.

Additional Considerations

- `SmartPoolHook` deliberately separates economic ownership from immediately deployable liquidity.
 - LP share previews and withdrawals are based on total managed assets, including vault positions valued through `convertToAssets`, while JIT deployment and indicative quoting use a capped, immediately withdrawable view. This design preserves LP claims over total value, but it also means operational issues in a configured vault can impair swap execution before they are reflected as a reduction in LP economic ownership.

- Reentrancy protection is layered and scope-specific. `SmartPoolHook` combines two distinct mechanisms:
 - User/admin entry points (`addLiquidity`, `removeLiquidity`, `setDistribution`, `setPoolLive`, `setExternalDeposits`, `refreshVaultApproval`) are guarded by both OpenZeppelin's transient `nonReentrant` and the `whenJITNotInProgress` modifier. The latter reads `JITLockable`'s global in-flight counter, so these functions revert if any pool on the hook is mid-JIT-cycle - closing cross-pool reentry from a callback (e.g. a vault hook fired during pool A's cycle) into these entry points on pool A or any other pool.
 - Swap callbacks rely on `JITLockable`'s per-pool lock: `_beforeSwap` calls `_enterJITLock(poolId)`, which reverts on re-entry into the same pool's cycle and increments the global counter; `_afterSwap` clears it after teardown. This prevents a callback from corrupting the in-flight pool's own deploy/settle/remove sequence.

Medium Severity

M-01 `removeLiquidity` Withdrawals Ignore ERC-6909 Claims At Payout

When an LP burns shares through `SmartPoolHook.removeLiquidity`, the payout flows through `MultiAssetVault.withdraw`, which first sizes the owed amounts via `_convertToAmounts` and then pays them out via `_pushAsset`.

There is an asymmetry between how the pool's balance is measured and how it is paid out. Specifically, measurement counts all three asset sources: `_convertToAmounts` reads the total managed balance through `_assetBalanceV4`, which sums `s.erc20`, `s.claims` (ERC-6909 claims on the `PoolManager`), and the ERC-4626 vault holdings. Thus, the share-to-amount conversion an LP is entitled to is computed including the pool's outstanding claims.

However, the hook only knows about two of those three sources. `_pushAsset` calls `_ensureERC20`, which sources liquidity exclusively from `s.erc20` and, for any shortfall, from the ERC-4626 vault capped at `vault.maxWithdraw`. It never redeems `s.claims`.

This contrasts with the JIT path, `_deployJIT`, which deliberately liquidates claims first (the cheap source) via `_redeemPoolClaims` before pulling only the residual shortfall from the vault. The reason `removeLiquidity` cannot simply do the same is that `_redeemPoolClaims` requires an active v4 unlock context, as it calls `poolManager.burn` and `poolManager.take`, and the `removeLiquidity` path does not open one.

Two potential consequences are as follows:

1. Some LP exits might be more expensive gas-wise. Currently, even when `s.erc20 + s.claims` fully cover the owed amount, `_ensureERC20` will route the shortfall through an ERC-4626 `vault.withdraw`, which involves a full external redemption with its own share burn and accounting, rather than the cheaper claim redemption used in `_deployJIT`. Claims are the cheapest liquidity source the pool has, and the withdrawal path never uses them.
2. Withdrawals may be bricked even if the pool is solvent. When `s.erc20` is insufficient and the vault is paused, capped, or utilization-constrained, `maxWithdraw` may return less than the shortfall and `_ensureERC20` reverts with `VaultLiquidityShortfall` (see `_ensureERC20`). This can happen even though the pool holds enough `s.claims`.

to satisfy the withdrawal, meaning the pool is economically solvent and the exit could be paid, but the LP is denied because the only shortfall source the code consults (the ERC-4626 vault) is throttled. Since `_assetBalanceV4` counts those same claims toward the LP's share value, the LP is told they own assets that the contract then refuses to pay out.

Consider redeeming `s.claims` on the withdrawal path before falling back to the ERC-4626 vault, mirroring the claims-first ordering already used in `_deployJIT`. Because claim redemption requires a v4 unlock context, this likely entails wrapping the `removeLiquidity` payout in a `poolManager.unlock` callback so that `_redeemPoolClaims` can run before `_ensureERC20`'s vault withdrawal. This would make typical exits cheaper when `s.erc20 + s.claims` already cover the owed amount, and would prevent solvent-but-vault-throttled pools from denying LP withdrawals.

Update: Resolved at commit [5a10577](#). The team stated:

removeLiquidity previously called `_withdraw` directly; the underlying `_ensureERC20` only inspects `s.erc20` and the vault when sourcing the LP's pro-rata payout. After a swap, `_depositAllToVaults` sweeps `s.erc20` to 0 and `afterSwap` mints positive hook deltas as ERC-6909 claims -- so the LP is owed an amount priced against $(erc20 + claims + vault)$ but `_ensureERC20` calls `vault.withdraw(amount)`, where `amount` includes the claim portion the vault doesn't actually hold. Bricks LP exits whenever the vault is even mildly capacity-bound; for non-vaulted pools the subtraction underflows.

Wrap `removeLiquidity` in `poolManager.unlock` and redeem both currencies' claims inside the callback before the withdraw math runs. `s.erc20` then includes the claim portion and `_ensureERC20`'s early-return covers it without an oversized `vault.withdraw` call.

Adds `unlockCallback` with `UnauthorizedCallback` guard (`msg.sender` must be the `PoolManager`). `PoolVault` and `MultiAssetVault` are untouched. Regression test exercises a `MockMorphoVaultV2` pool with an idle-reserve cap and confirms the unpatched baseline reverts with `WithdrawExceedsIdleReserves` while the patched flow completes cleanly.

M-02 Stale Transient Liquidity Slots Across Multiple Swaps in One Transaction Can Revert the Swap

To avoid storage reads during JIT teardown, SmartPoolHook records each bucket's deployed liquidity in transient storage. In `_deployBuckets` it writes a bucket's slot only when that bucket received `non-zero liquidity`. During teardown, `_removeJIT` reads each per-bucket slot back and, for any non-zero value, issues the inverse `modifyLiquidity`. Crucially, it deliberately does not clear the slots after reading them, relying on transient storage auto-clearing at end of transaction.

However, this reliance can be problematic in some cases, as transient storage clears at the end of the transaction, not at the end of each JIT cycle. When a single transaction performs more than one swap on the same pool, the slots written by an earlier swap's deploy survive into a later swap's teardown. Combined with the if non-zero liquidity write guard, a slot can be left holding a value from a previous cycle that the current cycle never refreshed.

Specifically, the failing scenario is as follows:

- First swap: In `_beforeSwap`, `_deployBuckets` deploys Bucket-0 with non-zero liquidity, so its slot is written. In `_afterSwap`, `_removeJIT` removes that position but leaves the slot populated.
- Second swap in the same transaction: The price has shifted, so Bucket-0's computed allocation is now 0. Because of the non-zero liquidity guard, `_deployBuckets` does not deploy Bucket-0 and does not overwrite its slot, so the slot still holds the stale value from the first swap.
- Second teardown. `_removeJIT` iterates all buckets, reads the stale non-zero value for Bucket-0, and tries to remove liquidity that was never deployed in this cycle. The inverse `modifyLiquidity` underflows the pool position and the whole transaction reverts.

Consider making the per-bucket transient slots fresh per JIT cycle rather than per transaction, for example by explicitly clearing or overwriting them during each cycle, to avoid reverting transactions that contain more than one swap on the same pool.

Update: Resolved at commit [04ad21f](#). The team stated:

`_removeJIT`'s per-bucket transient slots scope to the transaction, not the unlock, so multiple swaps on the same pool in one tx share state. If bucket *i* deployed `liq=100` in swap 1 and computes `liq=0` in swap 2 (price moved across its range), `deployBuckets` skips the slot write (guarded by `liq > 0`), leaving `slot[i]=100`

stale from swap 1. Without the clear, swap 2's `_removeJIT` would `tload` 100 and call `modifyLiquidity` with -100 on a position that doesn't exist, reverting the swap. While this is an edge case only triggered when a single transaction executes more than one swap on the pool, it puts the hook in an inconsistent state that causes a revert on otherwise valid behavior.

Clear each slot immediately after the `tload`. Costs ~ 100 gas per bucket per teardown, but removes a liveness bug that bricks any aggregator or multi-hop route through `SmartPoolHook` whenever a price move flips a bucket allocation to zero between swaps.

M-03 Fee-Charging ERC-4626 Vaults Break Share Accounting on Deposit and Withdrawal

The share accounting assumes the configured ERC-4626 vault charges no entry or exit fees. Pool holdings are valued gross, through `_assetBalanceV4`, which reads `vault.convertToAssets(shares)` and never subtracts any entry or exit fee. A vault that applies such a fee, where `previewDeposit` or `previewRedeem` differ from `convertToShares` or `convertToAssets`, is incompatible with the share accounting, and both LP actions are affected.

On withdrawal, `_withdraw` sizes the LP's payout with `_convertToAmounts`, which derives the owed amount from the gross balance. With an exit-fee vault the assets actually realizable are less than this, so the promised amount is overestimated. However, the payout path must nonetheless honor the overestimated amount, so the request can exceed what the vault reports as withdrawable and `revert with VaultLiquidityShortfall`. Since the gap is socialized across the remaining shares, this becomes a first-out-wins / last-out-loses race: early LPs redeem their full gross-valued claim, and the accumulated fee eventually bricks the withdrawals of those who exit later.

On deposit, `_deposit` mints shares against the gross `want` amount, so a deposit into an entry-fee vault succeeds: the LP is charged the full `want`, the hook receives the full `want`, but the vault credits strictly fewer shares since part of `want` is taken as an entry fee. The `TransferReceiptShortfall` guard does not catch this, as it only measures receipt at the hook, not the vault's net crediting. Furthermore, `_depositToVault` reconciles `_vaultShares` down to the actual shares minted, so the pool gross balance grows by less than the minted pool-shares imply. Existing shareholders are silently diluted, and no revert surfaces the loss.

The root cause is the same in both directions: the system treats `convertToAssets` and `convertToShares` as exact, while a fee-charging vault always delivers net. It promises gross value on the way out and books gross value on the way in.

Consider treating ERC-4626 vaults that apply an entry or exit fee as unsupported, and making that explicit through documentation or validation at pool configuration. Alternatively, consider handling such fees correctly so that share accounting remains consistent across `SmartPoolHook` and the associated vaults.

Update: Resolved at commit [f0f6fcb](#). The team stated:

PoolVault's design assumes feeless ERC-4626 vaults. Vaults that charge entry or exit fees are structurally incompatible:

1. JIT-cycle bleed. Every swap does `vault.withdraw` -> `swap` -> `vault.deposit`, so a 10bps+10bps round-trip vault drag bleeds 20bps of JIT notional per swap charged entirely to LPs, dwarfing the LP fee.

2. LP share-math socialization. `_assetBalanceV4` uses `convertToAssets` (gross per EIP-4626), but actual `vault.deposit`/`vault.withdraw` nets out the fee. The mismatch is socialized: depositors underpay (existing LPs subsidize), withdrawers over-extract (remaining LPs subsidize). Manifests as a first-out-wins / last-out-loses redemption race.

Detect at pool init via `_requireFeelessVault`: EIP-4626 specifies that `convertToShares`/`convertToAssets` MUST NOT factor in fees while `previewDeposit`/`previewRedeem` MUST. Probe asserts equality at `10**decimals`. Catches honest fee disclosures; deeply adversarial vaults that lie at preview level are out of trust scope (operators select curated vaults).

M-04 A Dust `_afterSwap` Deposit To Vault Can Revert an Entire Swap via `ZeroSharesMinted`

After a swap, the `_afterSwap` hook removes the JIT liquidity, resolves the deltas, and then deposits all remaining pool funds back into the ERC-4626 vaults via `_depositAllToVaults`. For each currency, `_depositAllToVault` calls `vault.deposit` and `reverts with ZeroSharesMinted` if the vault returns zero shares.

This fail-fast check is reasonable for a normal deposit, but it is also reached during swap teardown, where the leftover amount can be dust. When the remaining balance is small enough

that the vault mints zero shares for it (for example, a large swap against a low-liquidity pool that leaves only a tiny residual on one side), `vault.deposit` returns 0, `_depositAllToVault` reverts, and the revert propagates all the way up, failing the entire swap. The swap itself was otherwise valid; it is blocked only because the post-swap housekeeping cannot re-deposit a dust remainder.

In the `_afterSwap` re-deposit path, consider checking the predicted shares before depositing and skip the deposit when it would mint zero. The dust can remain tracked in `s.erc20` and be folded into the next deposit, letting the swap complete. This keeps the fail-fast `ZeroSharesMinted` behavior for genuine user-facing deposits while preventing a dust remainder from blocking swaps.

Update: Resolved at commit [6a256e0](#). The team stated:

Two related changes to PoolVault's vault-deposit handling:

1. Wrap `_depositAllToVault`'s `vault.deposit` in try/catch. The JIT `afterSwap` settlement no longer propagates vault-side reverts; a capped or paused vault (Morpho VaultV2 with full idle allocations, OZ ERC-4626 at `maxDeposit`, curator-rejected, etc.) would otherwise brick every swap on the pool. On failure the state is atomically reverted by the try block and we emit `VaultDepositSkipped` so operators can observe the misconfiguration. `ZeroSharesMinted` still propagates; that's an asset-loss failure mode distinct from a cap rejection. The LP-deposit path (`_depositToVault`) is intentionally left to revert directly; LP add operations should surface vault rejection to the user.

2. Drop the pre-credit + reconciliation pattern from both deposit paths. The pattern was defending against view-coherence inside vault-deposit callbacks, but PoolVault's entry points are locked during deposit (`nonReentrant` on LP path, JIT lock on JIT path), so the inconsistency window isn't actionable. The subtract-on-reconcile branch is also dead code now that `_requireFeelessVault` rejects fee-charging vaults at init.

M-05 Vault Deposit Cap Can Cause Swaps To Revert During `_afterSwap` Re-Deposit

After a swap, the `_afterSwap` hook removes the JIT liquidity, resolves the deltas, and then deposits all remaining pool funds back into the ERC-4626 vaults via `_depositAllToVaults`. For each currency, `_depositAllToVault` calls `vault.deposit(amount, address(this))` for the full tracked balance, with no check against the vault's deposit cap.

As per ERC-4626, `deposit` must revert when the amount exceeds `maxDeposit`. If a swap increases one currency's balance to the point where re-depositing it would exceed that vault's cap, `vault.deposit` reverts and the entire swap fails. This is not limited to a single failed swap. Once a vault is at or near its cap (for example because it is capped, paused, or has grown close to a supply ceiling), every swap that ends with more of that currency to re-deposit will keep hitting the cap and reverting. A vaulted pool can therefore be effectively bricked for swaps in that direction until the cap clears, driven entirely by the vault's state and not by anything wrong with the swap itself.

Consider making the post-swap re-deposit logic cap-aware so that leftover funds that cannot be deposited remain tracked as raw ERC-20 instead of reverting the swap because of a vault's `maxDeposit` cap.

Update: Resolved at commit [6a256e0](#). The team stated:

| Resolved as part of M-04 fix, same commit applies to both.

M-06 `sqrtPrice` Can Be Manipulated for Free in a Live-but-Unbootstrapped Pool

SmartPoolHook splits pool creation into two separate owner-only steps, `initializePool` and `bootstrap`, which may be performed in different transactions. `initializePool` calls `poolManager.initialize(key, config.sqrtPriceX96)` and immediately marks the pool live by setting `livePools[poolId] = true`. `bootstrap` seeds the pool's reserves and mints the first shares, establishing the share/asset ratio. Nothing forces these into the same transaction, so a pool can sit in a live-but-unbootstrapped state for an arbitrary window.

Meanwhile, `_beforeSwap` gates swaps only on liveness, [reverting with PoolNotLive](#) but never checking that the pool has been bootstrapped. When a swap hits an unbootstrapped pool, `_beforeSwap` still runs `_deployJIT`, which finds no reserves and [returns early](#), so no JIT liquidity is deployed and the swap proceeds against a v4 pool with zero liquidity. A swap against a zero-liquidity pool trades no tokens but walks the price straight to the caller's `sqrtPriceLimitX96`.

This lets an attacker move the pool's `sqrtPrice` to an arbitrary value at essentially no cost during the window between `initializePool` and `bootstrap`. When the owner subsequently bootstraps and the first real swap arrives, `_deployJIT` sizes and centers the bucket allocations around the [current slot price](#) and feeds that price into `_computeAllocations`. Because `bootstrap` fixes the share/asset ratio independently of the pool's `sqrtPrice`, the first JIT deployment is centered on the attacker-chosen,

manipulated price rather than on the true value of the freshly bootstrapped reserves. The attacker can then swap against this mispriced liquidity in the same block as the `bootstrap` and extract value from the pool's LPs.

Note that this free-manipulation primitive is specific to the unbootstrapped window: once reserves exist, every swap's `_beforeSwap` deploys real JIT liquidity before the swap executes, so a swap can no longer move price against zero liquidity.

Consider gating swaps on bootstrap, so that swaps cannot occur while the pool is empty, in order to remove the zero-liquidity swap window and prevent the first JIT deployment from being anchored to a manipulated `sqrPrice`.

Update: Resolved at commit [503e09d](#). The team stated:

`initializePool` previously set `livePools[poolId] = true` immediately, creating a window between `init` and the owner's bootstrap TX where a swap could land on a pool with `slot0.sqrPriceX96` set but zero LP shares. Such a swap deploys zero JIT liquidity yet may still shift `slot0`'s price, letting an attacker push the price away from the intended bootstrap value and extract on the first real swap after the owner bootstraps.

Move the live-flip to the end of `bootstrap` so the pool is live iff at least one share is outstanding. The existing `PoolNotLive` guard in `_beforeSwap` already covers the new state; no new error, hot-path check, or storage is needed. Owner can still pause/unpause via `setPoolLive` post-bootstrap.

Low Severity

L-01 Inconsistent Native Currency Handling in SettlementLib

The `SettlementLib` library is documented as a general-purpose delta settlement authority for ALF hooks that custody an `Inventory`. Its `resolveCurrency` function branches on `currency.isAddressZero()` although this branch is currently unreachable: the library's only consumers, `DualPoolHook` and its `DualPoolIncentivizedHook` subclass, reject native currencies during `initializePool`, so both `resolveCurrency` call sites always receive ERC-20 currencies.

Nonetheless, because `SettlementLib` is intended as a general-purpose settlement authority and the native branch is present, its implementation is expected to hold for native settlement as well. The branch deviates from the reference `DeltaResolver` implementation in v4-periphery, whose `__settle` function calls `poolManager.sync(currency)` before every settlement, including native, whereas `resolveCurrency` omits this sync call and invokes `settle{value: owed}()` directly. This is an inconsistency between the general-purpose intent of the library and its actual implementation. Specifically, `SettlementLib` would misattribute the settled amount to the wrong currency if any other currency were ever left synced but unsettled within the same unlock, which is the situation the unconditional sync in `DeltaResolver` is designed to prevent.

As a result, `SettlementLib` is neither cleanly minimal nor cleanly general: it carries a native-currency path that is dead code for every present consumer, while implementing that path in a way that is inconsistent with the reference settlement flow. This hurts readability and leaves a latent risk should the library later be reused by a hook that does support native currency.

Consider resolving the inconsistency in one of two ways: either remove the `isAddressZero` branch and narrow `SettlementLib` to ERC-20 settlement, matching the fact that all current consumers reject native currencies; or retain native support and add a `poolManager.sync(currency)` call before calling `settle{value}` so that the flow matches the `DeltaResolver` reference and remains correct for any future native-supporting consumer.

Update: Resolved at commit [bbd6563](#). The team stated:

*Added `poolManager.sync(currency)` before `settle{value: owed}` in the native branch of `resolveCurrency`, matching the ERC-20 branch and v4-periphery's `DeltaResolver`. We chose to **keep the library general** rather than delete the branch: it's documented as a general-purpose settlement authority, the native path is dead for all current consumers (so zero runtime impact today), and this closes the latent misattribution risk for any future native-supporting hook. Updated the class `NatSpec` accordingly.*

L-02 `initializePool` Accepts Dynamic-Fee Keys, Permanently Creating a Pool Where `SmartPoolHook` Cannot Operate Correctly

The `SmartPoolHook` is built on the [assumption](#) that the LP fee is static, but `initializePool` does not validate that `key.fee` encodes a static LP fee.

If the owner passes `key.fee = LPFeeLibrary.DYNAMIC_FEE_FLAG (0x800000)`, the call to `poolManager.initialize` succeeds and the pool is created in a state where the hook cannot function properly. For dynamic-fee keys, v4-core sets `slot0.lpFee = 0` at initialization and expects the hook to supply the live fee, either by returning an override from `beforeSwap` or by calling `poolManager.updateDynamicLPFee`. `SmartPoolHook` does neither: its `__beforeSwap` returns a zero fee override (it intends to charge the static `key.fee` natively), and the hook exposes no path to set or update the LP fee. The pool is therefore unrecoverable.

Consider rejecting dynamic-fee keys at the top of `initializePool`, before `poolManager.initialize` is reached. This would make the static-fee assumption explicit and fail fast on misconfiguration rather than silently creating an unrecoverable pool.

Update: Resolved at commit [f7fef6d](#). The team stated:

SmartPool's contract-level invariant ("pricing is fully static — set at deploy time via `PoolKey.fee` and never change") is documented but was not enforced. A caller passing `fee: LPFeeLibrary.DYNAMIC_FEE_FLAG (0x800000)` would slip past `initializePool` and land in `PoolManager.initialize`, which would treat the hook as the fee oracle for every swap, except the hook never updates fees and has no callback wired for it.

Add an explicit check at the boundary so the static-fee assumption is an enforced requirement, with a dedicated `DynamicFeeNotSupported` error that makes the rejection unambiguous in router/integration error surfaces.

L-03 Redeeming Accumulated Claims Within the Same Transaction Can Revert Later Swaps

When a swap leaves the hook with a positive net delta (the `PoolManager` owes the hook tokens), the teardown in `__resolveNetDeltaCurrency` does not immediately take the real ERC-20 funds. Instead it mints ERC-6909 claims and records them via `__recordClaims`, precisely because the swapper has not settled yet and the `PoolManager` may not hold the corresponding tokens during the in-flight transaction.

On the next swap, `__deployJIT` redeems the pool's claims for both currencies via `__redeemPoolClaims` as its first and cheapest liquidity source. `__redeemPoolClaims` burns the full claim balance and then takes the same amount of real ERC-20 out of the `PoolManager`.

However, a conflict arises if multiple swaps on the same pool are included in a single transaction:

- First swap: The hook ends with a positive delta and mints claims for it. These claims are backed only after the swapper settles — which happens at the end of the transaction, not now. So the `PoolManager` does not yet physically hold the tokens for these claims.
- Second swap (same transaction): `_deployJIT` calls `_redeemPoolClaims`, which tries to take the full claim balance — including the claims minted in the first swap — as real ERC-20 from the `PoolManager`. Because the first swapper has not settled yet, the `PoolManager`'s actual token balance may be short of what the `take` requires, so the transfer reverts and the second swap fails.

Thus, the redeem path assumes the claims it accumulated are immediately convertible to real ERC-20, but claims minted earlier in the same unsettled transaction are not yet backed by tokens in the `PoolManager`. This causes the later swap to revert even though everything is internally consistent and will settle correctly by the end of the transaction.

Consider capping `take` at the `PoolManager`'s currently available balance and sourcing the remainder from the vault in order to avoid failing swaps that contain more than one swap on the same pool within a single transaction.

Update: Resolved at commit [e73534d](#). The team stated:

Multiple swaps on the same SmartPool pool inside a single `PoolManager.unlock` reverted on the second swap. The first swap mints ERC-6909 claims for its positive delta, backed only by the swapper's input, which settles at the END of the unlock. The second swap's `_deployJIT` then (a) tried to `take()` those claims as real ERC-20 from the `PoolManager`, which may not yet physically hold them (although this is unlikely for high-cap stables as targeted by this hook), and (b) sized its deployment against `_effectiveAssets` (which counts the claims) and over-drew the vault for the claim portion the vault never held.

Two-part fix, both no-ops once claims are backed (the steady-state / separate-unlock path):

- `_redeemPoolClaims` caps the physical take at the `PoolManager`'s current balance of the currency and retains any unbacked remainder as recorded claims (redeemed on a later cycle once settled).

- `deployJIT` subtracts `unbackedClaims` from the deployable balance so it sizes liquidity only against what it can source now (vault + erc20 + claims we can take); any residual deploys on a future cycle.

L-04 Fixed `_decimalsOffset = 12` Makes 6-Decimal Pairs Impractical To Bootstrap

The `MultiAssetVault` share model uses a virtual-shares offset to reduce post-bootstrap inflation attacks. To ensure the bootstrapper's deposit is not materially diluted by the virtual position, `_bootstrap` enforces a floor of `100 * 10**_decimalsOffset(vaultId)` shares. Bootstrap `shares are computed` as `sqr((received0 * received1))` in raw base units.

Because `_decimalsOffset` is `not overridden` and always returns `12`, the minimum bootstrap size becomes extremely large for low-decimal pairs. For a 6/6 pool, satisfying `sqr((received0*received1)) >= 1e14` requires on the order of `1e14` base units per side (~100M tokens each) when bootstrapping symmetrically. If bootstrap cannot be completed, the vault remains unbootstrapped and future deposits and `share conversions revert` with `VaultNotBootstrapped`, effectively causing a pool-creation DoS for intended 6-decimal pairs. Even when bootstrapped at the minimum, the documented ~1% drift implies a large permanent bootstrapper loss at these required amounts.

Consider overriding `_decimalsOffset` per vault based on the underlying tokens' decimals, or otherwise normalizing `received0` and `received1` before computing the bootstrap floor, so that common 6-decimal pairs can be bootstrapped with realistic seed sizes. Alternatively, consider thoroughly documenting this limitation of the smart pool hook.

Update: Resolved at commit [919aee5](#). The team stated:

SmartPoolHook inherited MultiAssetVault's hardcoded `_decimalsOffset = 12`, even though the base NatSpec says subclasses handling low-decimal pairs SHOULD override it. With offset 12 the bootstrap floor is $100 \times 10^{12} = 1e14$ shares, so a 6-decimal stablecoin pair (USDC/USDT) needed ~100M tokens/side to bootstrap and reverted `BootstrapTooSmall` below that.

Override `_decimalsOffset` with a per-pool value cached at `initializePool`:

- `clamp((d0 + d1) / 2 - 6, 6, 12)`, read from the tokens' `decimals()` (fallback 18). An 18/18 pool stays 12 (same behavior as before the fix); 6/6 becomes 6 (~100-token min seed, 1e6 virtual-share defense). The drift at the

floor is ~1% regardless of offset by construction, so the offset only sets the absolute min seed and defense strength, which move together. Expose `decimalsOffset(poolId)` for off-chain bootstrap sizing.

L-05 `renounceOwnership` Allows Permanent Loss Of Administrative Control

The `SmartPoolBase` contract inherits `Ownable2Step` without overriding the inherited `renounceOwnership` function. As a result, the owner can transfer ownership to `address(0)` and permanently lock out every `onlyOwner` entry point, including `initializePool`, `setPoolLive`, `setActiveTick`, and `setAuthorizedLP`. Once ownership is renounced, no new pools can be created, the active tick and authorized-LP allowlist are frozen at their current state, and recovery requires redeploying the hook, which is non-trivial because the hook address encodes permission bits via `CREATE2` mining and every pool referencing the old hook becomes orphaned.

Consider overriding `renounceOwnership` in `SmartPoolBase` to revert with a dedicated error, matching the standard OpenZeppelin pattern for contracts where administrative recovery is load-bearing.

Update: Resolved at commit [d8e7721](#). The team stated:

Both bases inherit `Ownable2Step` without overriding the inherited `renounceOwnership`. Calling it would transfer ownership to `address(0)` and permanently brick every `onlyOwner` entry point; for `SpreadQuoter` that's `initializePool`, `setPoolLive`, `setActiveTick`, and `setAuthorizedLP`; for `SmartPool` also `bootstrap`, `setExternalDeposits`, `setDistribution`, and `refreshVaultApproval`. Recovery would require redeploying the hook at a new flag-mined address and orphaning every existing pool.

Override `renounceOwnership` in both bases to revert with a dedicated `RenounceOwnershipDisabled` error. Mirrors the standard OZ pattern for contracts where the operator is the single trust principal and administrative recovery is load-bearing. Use `Ownable2Step.transferOwnership` to rotate the operator address; renunciation is not a supported lifecycle operation for these hooks.

L-06 Documentation Does Not Match Implementation in Several Places

Several statements in [the SmartPoolHook documentation](#) and the related NatSpec describe behavior that the shipped code does not implement. None of these are exploitable on their own, but they misinform integrators and LPs about the trust model and runtime behavior, which can lead to incorrect integration assumptions. The following are examples of the outdated documentation:

- Owner is described as immutable, but ownership is transferable. The docs and constructor NatSpec state that "the owner is immutable and cannot be transferred," yet [SmartPoolHook](#) inherits [Ownable2Step](#) via [SmartPoolBase](#) and does not disable transfers — ownership can be moved post-deployment via the [transferOwnership](#) / [acceptOwnership](#) two-step flow.
- Paused pools are described as no-op, but they revert. The docs state that when a pool is not live, [beforeSwap](#) returns a zero delta and lets the swap proceed. In practice, [SmartPoolHook._beforeSwap](#) [reverts with PoolNotLive](#) when [livePools\[poolId\]](#) is false, which bubbles up and aborts the entire (potentially multi-leg) swap rather than no-op'ing.
- Fee-on-transfer / rebasing tokens are described as handled, but they are silently accepted and later unswappable. The docs claim inbound transfers are "measured to reject fee-on-transfer or rebasing shortfalls." [PoolVault._pullAsset](#) only measures the received amount and credits the reduced figure — it does not reject the shortfall. Such a pool can be deposited into but cannot process swaps, since [_resolveNetDeltaCurrency](#) settlement falls short of the recorded delta and [reverts](#).
- [getIndicativeQuote](#) is described as price-limit bounded, but it is unbounded. The docs describe the indicative quote path as "a single SwapMath.computeSwapStep bounded by the caller's price limit." However, [getIndicativeQuote](#) accepts no price-limit parameter and hard-codes the TickMath extremes ([MIN_SQRT_PRICE + 1](#) / [MAX_SQRT_PRICE - 1](#)) before delegating to [_simulateIndicative](#), so the simulation is not bounded by any caller-supplied price — the price-bounded variant is the separate [swapToPrice](#) function.

Consider reconciling the documentation and NatSpec with the shipped behavior in each case. Where the discrepancy reflects an intended design (e.g. transferable ownership, revert-on-pause), update the docs to match the code; where the documented behavior is the intended one (e.g. rejecting FoT/rebasing shortfalls), align the implementation instead — for example, by reverting in [_pullAsset](#) when the received amount is below the requested amount, while still

documenting that FoT/rebasing tokens are unsupported since a deposit-time check cannot catch fees or rebases enabled after deposit.

Update: Resolved at commit [b1a0299](#). The team stated:

Addresses a docs-vs-implementation review. Three of the four reported items were already corrected (pause reverts PoolNotLive, markdown owner section, etc.); this fixes the rest:

- Owner NatSpec (SmartPoolHook constructor) said the owner is immutable, but SmartPoolBase uses `Ownable2Step`. Corrected to: transferable via `transferOwnership/acceptOwnership`, `renounceOwnership` disabled.
- `getIndicativeQuote` was documented as bounded by the caller's price limit; it actually runs `computeSwapStep` across the full tick range. Only `swapToPrice` is caller-bounded. Clarified in the deep dive (markdown + Notion).
- Fee-on-transfer/rebasing tokens were documented as rejected, but bootstrap silently accepted under-receipt (minting `sqrt(received)`), seeding a pool whose recorded balances exceed what the hook can settle. `PoolVault._pullAsset` now reverts `TransferReceiptShortfall` on `received < want` for BOTH bootstrap and `addLiquidity`; docs clarified that this is a deposit-time check and cannot catch post-deposit fee/rebase changes.

L-07 Donated Vault Shares Inflate the Pro-Rata Denominator and Suppress Reported Effective Liquidity

`PoolVault` exposes a cap-aware view of immediately deployable assets through `_effectiveBalance`, which is consumed by `SmartPoolHook.getEffectiveLiquidity` (the routing and indicative-quote view) and by the JIT deploy path that sizes liquidity against `_effectiveAssets`. For vaulted pools, `_effectiveBalance` caps each pool's vault contribution at a pro-rata share of `vault.maxWithdraw(address(this))`, so that callers asking "what can be deployed right now" see the truth even when the vault is paused, capped, or utilization-constrained.

The pro-rata scaling uses `totalHookShares = IERC20(address(vault)).balanceOf(address(this))` as its denominator: `poolMax = globalMax * shares / totalHookShares`. Because ERC-4626 shares are ordinary ERC-20 tokens, any external account can transfer shares directly to the hook address without

going through deposit accounting. This increases `totalHookShares` while leaving the protocol-tracked `_vaultShares[poolId][currency]` unchanged. When the vault is in a withdrawal-capped state, so that `maxWithdraw(address(this))` does not grow proportionally with the extra shares, the inflated denominator shrinks `poolMax`, and therefore `_effectiveBalance`, for every pool sharing that vault.

The condition that makes this exploitable is intrinsic to the design rather than an edge case. The `maxWithdraw` cap exists precisely because the vault is a shared, public protocol vault whose withdrawable amount can be constrained by unrelated third-party activity. An attacker can acquire shares on the open market and donate them to the hook. Relying on `balanceOf` for internal accounting is the well-known token-donation danger, and it is only ever inflated, never deflated, since `sum(_vaultShares) == balanceOf(address(this))` holds exactly in the absence of donations.

No funds are at risk and the distortion is conservative in direction: `balanceOf` can only be inflated, so `_effectiveBalance` is only ever under-reported, never over-reported, and there is no over-deployment or solvency impact. The effect is a griefing vector that degrades indicative-quote quality and shrinks JIT depth for all pools on the affected vault, bounded to the vault's capped windows (it self-heals once the vault uncaps, since `maxWithdraw` then grows with the donated shares and the inflation cancels out). The attacker also pays a real and likely unrecoverable cost: donated shares land outside `_vaultShares`, so the protocol never withdraws them and no sweep path exists, and moving `poolMax` meaningfully requires donating on the order of the existing tracked total.

Consider tracking the total hook-held vault shares through internal accounting, for example through a per-vault `totalTrackedShares` keyed by vault address and updated alongside `_vaultShares` on every deposit and withdrawal, and using that as the pro-rata denominator instead of `IERC20(address(vault)).balanceOf(address(this))`. Additionally, consider an explicit handling path for unsolicited share transfers (for example a sweep or redistribution of any balance in excess of the tracked total) so that donations cannot strand value in the hook.

Update: Resolved at commit [2feb07a](#). It was further identified that vaults like MorphoV2 vault may cause integration issues as their `maxWithdraw` returns 0. The fix relinquishes `maxWithdraw` + pro rata calculation - for vault with withdraw caps - and replaces with `previewRedeem` for effective assets calculation. As a trade off, a pool having `holdings > withdraw cap` of the vault would subsequently revert swaps and fail gracefully since `vault.withdraw` in `_ensureERC20` would revert. This trade-off was considered an acceptable limitation of the current model.

The team stated:

Addressed incidentally while implementing Morpho vault support. Adapted PoolVault and MultiAssetVault so vaults that diverge from textbook ERC-4626 (notably Morpho VaultV2) can back a SmartPoolHook pool.

L-08 `getIndicativeQuote` May Overstate Output Beyond Pool-Deliverable Reserves

The `getIndicativeQuote` function of the `SmartPoolHook` contract produces a price estimate by running a single constant-liquidity swap step toward the `maximum or minimum tick price`, using the pool's current in-range bucket liquidity. For an order the pool can serve in full, this estimate is accurate. The estimate is then consumed by the `ALFMultiplexer`: it is used to rank candidate quoters, and, when a caller sets `strictTolerancePips`, it becomes the baseline against which the executed amount is compared in `_beforeSwap`.

However, the single-step computation extrapolates the in-range liquidity as a constant curve all the way to the price limit. It does not account for the fact that real liquidity is confined to the configured bucket ranges and is exhausted once the price moves beyond them. For an order larger than the pool can deliver, the estimate therefore reports more output than the pool actually holds. For exact-input orders, the overestimation can reach approximately twice the deliverable amount on a balanced single-bucket pool, and several times the deliverable amount on a skewed multi-bucket distribution where only a fraction of the nominal reserves is reachable in a single direction.

This overestimation has two consequences. First, a quoter that cannot fully serve an order can report the most attractive estimate and be ranked ahead of all quoters. Second, when `strictTolerancePips` is enabled, the inflated estimate becomes an unreachable baseline, so a swap that executes within otherwise acceptable bounds can still `revert with QuoteDeviation`, denying an otherwise valid fill. No funds are at risk. The impact is limited to degraded quote quality and to a liveness failure on opt-in strict-tolerance swaps, and the strict-tolerance path is disabled by default.

Consider capping the indicative output at the pool's deliverable reserves so that the estimate can never exceed the amount the pool could actually pay out, and excluding such extrapolated estimates from the strict-tolerance baseline. Where capping at total reserves still overstates the amount a single swap can deliver for highly skewed distributions, it is advisable to document this limitation so that integrators relying on strict tolerance understand that sufficiently large orders against such pools may revert.

Update: Resolve in commits [5fc2a4b](#) and [f2abf70](#), which cap the indicative output at the pool's deliverable reserves and bound the multiplexer's strict-tolerance baseline accordingly. After the cap, the estimate matches the deliverable amount for balanced distributions. A residual overestimation may remain for skewed or one-sided bucket distributions, where the cap at the pool's total output reserve still exceeds the amount a single swap can deliver in one direction. Overestimation grows larger as the skew becomes more extreme. This residual is acknowledged as an accepted design limitation, with binding slippage protection left to the caller or router.

Notes & Additional Information

N-01 Duplicate Code

In `PoolVault.sol` the `_pullAsset` function dispatches the flow to `_depositToVault` on whether a vault is configured for `(poolId, currency)`. However, the no-vault branch is already covered inside `_depositToVault` itself, which performs the exact same `_state.erc20` accumulation when `address(vault) == address(0)`. The if-else in `_pullAsset` is therefore redundant — both branches collapse to a single unconditional `_depositToVault(poolId, currency, received)` call.

Consider replacing the if-else block with a single call to `_depositToVault` to avoid duplicated logic that must be kept in sync across two locations.

Update: Resolved at commit [17a390a](#). The team stated:

Replace the conditional with a single unconditional `_depositToVault` call so the ledger-vs-vault routing lives in one place.

N-02 Unnamed Magic Number in `SmartPoolHook`

The literal `10_000` in `SmartPoolHook` is the basis-points denominator for bucket distribution weights — the total that all bucket `weightBps` values must sum to, and the divisor used when pro-rating balances per bucket. It is used as a bare literal in several places. As a repeated bare literal it reduces readability and invites divergence if one occurrence is changed without the others.

Consider extracting it into a single named constant (for example `WEIGHT_DENOMINATOR = 10_000` or `TOTAL_WEIGHT_BPS = 10_000`) and referencing it at each use site, including in the NatSpec that documents the "weights must sum to 10_000" requirement.

Update: Resolved at commit [389e658](#). The team stated:

Replace the literal 10_000 basis-points denominator with a named constant `TOTAL_WEIGHT_BPS` at all five code sites

N-03 Vault Addresses Are Permanently Assigned With No Emergency Revocation Path

In `SmartPoolHook`, `initializePool` assigns vault addresses to `vaults[poolId][currency0]` and `vaults[poolId][currency1]` and calls `_approveVault` to grant each vault a `type(uint256).max` token allowance. There is no subsequent write path to `vaults[poolId][currency]`. The `refreshVaultApproval` function can reset the allowance to `type(uint256).max` after consumption, but it cannot revoke the vault assignment itself.

If a configured vault is paused, exploited, or upgraded by its controller after pool initialization, the hook has no mechanism to isolate or remove the exposure. More critically, the hook-wide approval is not scoped per pool: a compromised vault for currency X can call `transferFrom` on the hook and drain the hook's entire ERC-20 balance of X, including amounts attributed to pools that do not use that vault. The NatSpec in `_depositToVault` acknowledges this cross-pool exposure but frames it as an acceptable vault-trust trade-off. There is no on-chain emergency stop between a vault compromise and the draining of the hook's full currency balance.

Consider adding an owner-restricted function that zeroes the token allowance to a specified vault, pauses the affected pool, disables external deposits, and simultaneously tries to withdraw the funds back to the hook.

Update: Resolved at commit [34dc87e](#). The team stated:

Adds an owner-only atomic emergency action for a suspect vault. The (pool, currency) - > vault binding is immutable, but the standing max allowance was previously irrevocable and assets already inside the vault had no rescue path.

Added `emergencyRevokeVault(key)`, which performs the following operations in order:

1. pauses the pool to block swaps immediately, which effectively prevents the pool from interacting with the vault for JIT purposes;
 2. disables external deposits because `addLiquidity` isn't liveness-gated, so without this an external depositor could re-arm the allowance via the deposit path even while the pool is paused, which bypasses the revocation;
 3. zeroes vault allowances (`_revokeVaultApproval`) to stop a compromised vault from draining the hook's raw ERC20 balance via `transferFrom`; and
 4. makes a best-effort attempt to redeem the pool's vault shares back to raw ERC20, which is economically neutral from the pool's POV (`getReserves` unchanged) but ensures recovery of the assets (insofar as the vault has the ability to redeem our shares)
- Note that `removeLiquidity` stays open so LPs can still exit.

N-04 Allowance Mutation Bypasses

`InventoryLib` in `DualPoolHook.refreshVaultApproval`

In the type-centered model, mutation of the hook's ERC-4626 vault allowances is encapsulated in `InventoryLib`, which exposes `approveVault` (idempotent max approval at bind time), `revokeVaultApproval` (emergency zeroing), and `ensureVaultAllowance` (refresh to at least a required amount). The `PoolVault` base routes through these helpers via its `_approveVault` and `_revokeVaultApproval` wrappers, and the `Inventory` type documents allowance handling as part of its responsibilities.

The `refreshVaultApproval` function in `DualPoolHook` is the only allowance mutation that does not follow this pattern. Instead of delegating to `InventoryLib`, it calls `SafeERC20.forceApprove` on the token directly.

This is the sole reason `DualPoolHook` declares `using SafeERC20 for IERC20`, and it splits the allowance concern between the hook and the library that otherwise owns it, weakening the consistency of the type-centered design.

Consider routing `refreshVaultApproval` through `InventoryLib`, either by delegating to `ensureVaultAllowance` with `type(uint256).max` via a `PoolVault` wrapper, or by adding a dedicated refresh helper to `InventoryLib` so that all allowance mutations flow through a single library and the direct `SafeERC20` dependency can be removed from the hook.

Update: Resolved at commit [e959aad](#).

N-05 Overloaded "bucket" Terminology

The term "bucket" is used throughout the ALF module to name two unrelated concepts:

- In `Distribution` (and `ActiveLiquidity`), a bucket is a `LiquidityBucket`: a tick range paired with a weight, representing one slice of the JIT liquidity distribution deployed during swaps.
- In `Inventory` (and the code that consumes it: `InventoryLib`, `SettlementLib`, `PoolVault`, and `DualPoolHook._bucket`), a bucket is a `bytes32` accounting key derived from a `(poolId, currency)` pair. It identifies a pool-currency's custody partition: the ERC-4626 vault bound to it, the vault shares it owns, and its raw ERC-20 and ERC-6909 claim balances.

These are distinct in type (`bytes32` versus `LiquidityBucket`), in key (a pool-currency pair versus a tick range), and in lifecycle (long-lived custody accounting versus per-swap liquidity placement). Because `DualPoolHook` operates on both types, the shared name forces the reader to infer from context which concept is meant in each location, reducing readability and increasing the risk of future maintenance errors.

Consider reserving the term "bucket" for a single concept and renaming the other. Since a tick-range slice is the more conventional use of the word in this context, renaming the `Inventory` accounting key to a distinct term such as "partition" throughout `Inventory`, `InventoryLib`, `SettlementLib`, `PoolVault`, and `DualPoolHook` would confine each term to one meaning and make the code unambiguous.

Update: Resolved at commit [221bab3](#). The team stated:

Renamed the `bytes32 (poolId, currency)` custody accounting key from "bucket" to "partition" across `Inventory`, `InventoryLib`, `SettlementLib`, `PoolVault` and `DualPoolHook`'s delta-resolution path and their unit tests. "bucket" is now reserved for the distribution `LiquidityBucket` sense (`MAX_BUCKETS`, `Distribution/ActiveLiquidity`, the JIT deploy path, `ALFMultiplexer/SwapSimulator`).

N-06 Inconsistent Parameter Order in `earned`

The `Rewards` library exposes three free functions that each take both the total share supply and a user's share balance. Two of them declare these parameters in the order `(totalSupply, userShares)`. The `earned` function, however, declares them in the reverse order `(userShares, totalSupply)`.

Because both parameters are of type `uint256`, the compiler cannot detect a call site that passes them in the wrong order, and the two conventions differ only in a way that is easy to overlook. The current consumer, `DualPoolIncentivizedHook`, does call each function with the correct order for its respective signature, so there is no present bug. The concern is that the divergence is an error-prone trap for future callers and reduces the readability of the interface.

Consider aligning `earned` with `checkpoint` and `claim` by declaring its parameters in the `(totalSupply, userShares)` order, so that all functions in the `Rewards` library share a single, consistent convention.

Update: Resolved at commit [e3b21f7](#).

Conclusion

The Active Liquidity Framework SmartPool stack implements a Uniswap v4 hook that supplies just-in-time liquidity around a pool's active tick while holding idle inventory in external ERC-4626 vaults, together with the ALF Multiplexer, which resolves quotes through a tiered waterfall over candidate sources. OpenZeppelin audited the SmartPool hook, its liquidity, vault, and share-accounting libraries, and the Multiplexer quoting components.

The framework separates quote simulation, just-in-time liquidity deployment, and vault custody, and its indicative quoting tracks executed swaps closely in the regimes it is designed for. Much of the residual risk concentrates in two areas. The first is the behavior of integrated ERC-4626 vaults, where the implementation assumes that deposits and withdrawals preserve credited value and remain available on demand. The second is operations that span a single transaction, where transient liquidity accounting and same-transaction claim redemption can interact in unexpected ways.

The most severe issue identified, a medium-severity finding, allowed the price of a live but unbootstrapped pool to be moved at no cost. Several medium-severity issues stemmed from vault assumptions, including fee-charging vaults breaking share accounting and deposit caps or dust re-deposits reverting otherwise valid swaps, while lower-severity findings addressed pool configuration, accounting precision and documentation errors.

OpenZeppelin thanks the Uniswap team for their cooperation and responsiveness throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.